

Historia de usuario M6.1S4

Administración de Relaciones y Consultas de Negocio de "Eventify"

Objetivo de la Historia de usuario

Evolucionar la arquitectura de "Eventify" implementando un modelo relacional complejo, optimizando el acceso a datos mediante proyecciones avanzadas y proactivas, y garantizando una gestión profesional del esquema mediante migraciones versionadas y auditoría SQL en tiempo real.

Contexto de Negocio

El éxito de "Eventify" depende de su capacidad para organizar información masiva de forma rápida y confiable. El negocio requiere que cada evento esté vinculado obligatoriamente a una sede física y clasificado en múltiples categorías temáticas. Por seguridad operativa y cumplimiento de auditoría, queda prohibida la eliminación física de registros; se debe mantener la trazabilidad histórica mediante un sistema de borrado lógico. El sistema debe ser capaz de servir catálogos de miles de registros sin degradar el tiempo de respuesta, utilizando estrategias de carga de datos de nivel empresarial.

Detalle de la HU

- **Como:** Coordinador de eventos/Administrador
- **Quiero:** Gestionar eventos interconectados con lugares y categorías, navegar por catálogos paginados y filtrados, y contar con una base de datos versionada y auditable.
- **Para:** Garantizar un catálogo profesional, optimizado y con integridad total de la información de negocio.

TASK 1

Modelado de Relaciones e Integridad de Datos (Soft Delete)

1. **Evolución del Modelo Geográfico:** Ampliar la entidad Lugar (Venue) añadiendo el atributo ciudad (city). Este campo es obligatorio y debe permitir segmentaciones precisas.
2. **Gestión del Ciclo de Vida (Soft Delete):** Implementar el borrado lógico en la entidad Evento (Event) mediante un atributo activo (active/isActive).
 - **Regla técnica:** El borrado debe ser transparente. Se exige un filtro global automático (investiga @SQLRestriction) para que las consultas nunca retornen registros inactivos. La "eliminación" debe invocar un método de actualización de estado (softDelete/deactivate).
3. **Nueva Entidad de Clasificación:** Crear la entidad Categoría (Category) con atributos nombre (name) y descripción (description).
 - **Ejemplos de categorías:** Conciertos (Concerts), Talleres (Workshops), Conferencias (Conferences), Deportes (Sports), Gastronomía (Gastronomy), Festivales (Festivals), Teatro (Theater).

4. **Estructura Relacional Profesional:** Establecer una relación Many-to-One obligatoria entre Event y Venue, y una relación Many-to-Many entre Event y Category.
 - **Regla técnica:** Definir nombres de tablas intermedias (ej. events_categories) y columnas de unión (event_id, category_id) de forma explícita.
5. **Integridad de Persistencia:** Configurar el guardado en cascada para que al persistir un evento se gestionen automáticamente sus vínculos relacionales.

TASK 2

Ingeniería de Rendimiento y Acceso a Datos

1. **Optimización de Memoria con Records:** Implementar el Record EventoResumenDTO (EventSummaryDTO) para listados masivos.
 - **Regla técnica:** El DTO debe "aplanar" la información (Nombre Evento, Fecha, Nombre Lugar, Ciudad) mediante un constructor en la consulta JPQL. Se prohíbe el uso de entidades pesadas en los listados.
2. **Estrategia de Carga con Slice:** Utilizar la interfaz Slice<T> para recuperar fragmentos de datos, optimizando la latencia al evitar el conteo total de registros.
3. **Resolución del Problema N+1:** Asegurar que la carga de relaciones (Lugar y Categorías) se realice en una sola operación de base de datos.
 - **Regla técnica:** Se exige el uso de @EntityGraph o JOIN FETCH para garantizar la carga ansiosa y eficiente del catálogo.
4. **Auditoría SQL y Observabilidad:** Configurar el sistema para registrar en consola las sentencias SQL formateadas y los valores de los parámetros (BasicBinder=TRACE).

TASK 3

Motor de Búsqueda Avanzado y Evolución de Swagger

1. **Motor de Consultas Filtradas:** Implementar métodos en el repositorio para la búsqueda por ciudad, categoría, capacidad y rango de fechas (ej. findByCity, findByDateBetween).
2. **Ordenamiento y Consistencia:** Garantizar que todos los listados se entreguen ordenados cronológicamente (OrderByDateDesc).
3. **Evolución de la Documentación Técnica (Swagger):**
 - **Actualización del Modelo:** Reflejar la transición de la API hacia el uso de Records (EventSummaryDTO).
 - **Documentación de Filtros:** Anotar el controlador para explicar cómo los nuevos parámetros relacionales impactan en el resultado.
 - **Trazabilidad de Reglas:** Asegurar que la documentación evolucione a la par del código, describiendo el impacto del borrado lógico mediante anotaciones @Operation y @Parameter.

TASK 4

Infraestructura de Datos e Interfaz de Usuario

1. **Configuración de Evolución de Base de Datos (Flyway):**
 - **Gestión de Dependencias:** Añadir la dependencia de flyway-core al proyecto.
 - **Configuración del Entorno:** Desactivar la generación automática de tablas de Hibernate (spring.jpa.hibernate.ddl-auto=none o validate) para delegar el control total a Flyway. Asegurar que spring.flyway.enabled=true.
2. **Ciclo de Migraciones Manuales:** Implementar 3 scripts SQL secuenciales en src/main/resources/db/migration:
 - **V1:** Estructura inicial de tablas.
 - **V2:** Evolución relacional (nuevos campos y tablas de categorías).
 - **V3:** Seed de datos masivo con al menos 200 registros variados.
3. **Formularios Relacionales Dinámicos:** Actualizar la interfaz Thymeleaf para permitir la asignación de Venue (desplegable) y Category (checkboxes).
4. **Navegación Paginada y Persistente:** Implementar una barra de navegación (Anterior/Siguiente) que mantenga los filtros activos en la URL y se deshabilite automáticamente según el estado del Slice.

Criterios de aceptación

Escenario 1: Persistencia Relacional y Trazabilidad SQL

- **Dado** el registro de un nuevo Event con Venue y Category asignados.
- **Cuando** se guarda la información y se revisa la consola de logs.
- **Entonces** el sistema debe mostrar el INSERT principal y los vínculos relacionales con sus parámetros reales visibles.

Escenario 2: Eficiencia de Consulta y Ordenamiento

- **Dado** un catálogo de 200 registros en la base de datos.
- **Cuando** se carga la página administrativa.
- **Entonces** se debe visualizar UNA sola consulta SQL optimizada (JOIN) y los registros deben aparecer ordenados por fecha de forma descendente.

Escenario 3: Navegación y Persistencia de Contexto

- **Dado** un usuario que realiza una búsqueda por categoría y ciudad.
- **Cuando** el usuario avanza de página utilizando los controles de navegación visual.

- **Entonces** el sistema debe mostrar los resultados correspondientes a esa búsqueda en la nueva página, manteniendo los parámetros de filtro en la URL.

Escenario 4: Integridad del Soft Delete y Evolución de Swagger

- **Dado** el acceso a Swagger UI.
- **Cuando** se consulta la documentación de los endpoints de catálogo y búsqueda.
- **Entonces** la interfaz debe reflejar los esquemas actualizados de los Records, los nuevos parámetros relacionales y las descripciones funcionales del borrado lógico.

Escenario 5: Validación de Evolución de Esquema (Flyway)

- **Dado** el arranque de la aplicación con una base de datos limpia y ddl-auto=none.
- **Cuando** Flyway procesa los scripts V1, V2 y V3.
- **Entonces** la base de datos debe quedar poblada con 200 registros y la estructura relacional completa sin errores de migración.

Escenario 6: Robustez de Filtros (Búsqueda Parcial e Insensible)

- **Dado** un evento llamado "Concierto de ROCK" en la ciudad de "Bogotá".
- **Cuando** el usuario busca por categoría "rock" (en minúsculas) o ciudad "bog" (parcial).
- **Entonces** el sistema debe encontrar y mostrar el registro, demostrando que la búsqueda no es rígida ni sensible a mayúsculas.